Software Entwurf 4
Übung 2

Sommersemester 2026

HSD FH-HAGENBERG

(1) Name: Simon Offenberger
Mat.Nr.: S2410306027

(2) Name: Jakob Renner
Mat.Nr.: S2410306009
Übungsgruppe: 1
Aufwand in Mh: 10

Abgabetermin: KW 15
Punkte: _____
korrigiert: _____

Beispiel 1 (10 Punkte) Parallelisierung: Ihr Logistik-Unternehmen verfügt über 8 LKWs, die in acht verschiedenen Niederlassungen stationiert sind. Für einen Hilfsgütertransport wurden ihnen an allen Standorten eine Vielzahl an Gütern abgegeben - viel mehr, als Sie auf einem LKW transportieren können. Sie stehen vor dem Problem, nun jene Güter auszuwählen, die bei der begrenzten Ladekapazität ihres LKWs den maximalen Nutzen für das Katastrophengebiet haben (maximaler Wert bei geringstem Gewicht).

Entwickeln Sie für ihre Zentrale eine Software, die diese Optimierung für die 8 LKW durchführt. Jedes Gut hat ein Gewicht und einen Nutzen (bzw. Wert). Ihr LKW hat eine maximale Nutzlast. Um herauszufinden, welche Güter Sie mitnehmen, müssen Sie alle Güterkombinationen ausprobieren - für jeden LKW getrennt.

Lassen Sie sich vom vorhandenen Programm `Optimierung.cpp` inspirieren und machen Sie sich mit der Funktionsweise des Algorithmus vertraut.

a) Entwickeln Sie eine sequenzielle Lösung, wo Sie alle 8 LKW-Ladungen hintereinander berechnen. Messen Sie die Zeitdauer und kommentieren Sie diese.

b) Führen Sie zur Beschleunigung der Berechnung eine Parallelisierung durch, in dem Sie jeden einzelnen LKW jeweils in einem eigenen Thread (insges. 8 Workerthreads) rechnen. Welchen Speedup können Sie auf ihrer Hardware erreichen? Dokumentieren Sie zusammen mit der verwendeten Hardware die erzielten Ergebnisse.

Anmerkungen:

- Legen Sie für jeden LKW eine eigene ASCII-Datei an, die Sie jeweils einlesen. Die Datei speichert die LKW-Nutzlast sowie Gewicht & Nutzen alle Güter. (Anm.: Nehmen Sie - vereinfachend - für jeden LKW die gleiche Datei.)
- Transportieren Sie rund 40 - 50 Güter pro LKW. Sollte ihr Rechner zu langsam sein, reduzieren Sie individuell die Anzahl, damit Sie akzeptable Laufzeiten bekommen.
- Legen Sie die Nutzlast so fest, dass Sie rund 50% der Güter transportieren können.
- Sie haben nur eine Fahrt, dürfen den LKW nicht überladen, und keine Güter zwischen den Standorten tauschen.

Hinweise zur Ausarbeitung:

- Führen Sie die Geschwindigkeitstests jeweils mit der **Release-Version** durch.
- Übersetzen Sie ihr paralleles Programm mit den **Multithreaded Bibilotheken** (Project Properties: C/C++ -> Code Generation -> Runtime Library: Multi-threaded DLL).
- Die **Terminierung von Threads** können Sie mit den API-Calls `WaitForSingleObject()` bzw. `WaitForMultipleObjects()` abwarten (siehe dazu die MSDN-Hilfe).
- Verwenden Sie die **Threadklasse** aus der Übung!

Beispiel 2 (14 Punkte) Admin-Tool: Um sich mit der Win32/64-Schnittstelle besser vertraut zu machen, implementieren Sie ein Administrationswerkzeug, das über folgende Funktionen verfügt:

1 Applikationen starten: Via Menüauswahl im Testtreiber können zumindest der *Editor* (`notepad.exe`) und der *Taschenrechner* (`calc.exe`) gestartet werden - auch mehrmals.

2 Prozesse auflisten: Alle Prozesse im System werden aufgelistet (sofern die Zugriffrechte existieren). Die Formatierung ist optional (`setw()`, `setfill()`, `left`, etc), zB:

```
Process        PID      BasePriority   Thread Count
-------------------------------------------------------
calc.exe       123      normal             1
notepad.exe    233      high               1
Mcshield.exe   215      low               18
calc.exe       317      low                1
Acrobat.exe    127      normal             4
...
```

3 Bestimmte Prozesse terminieren: Die über die PID angegebene Applikation wird terminiert, sofern die Rechte bestehen.

4 Systeminformation anzeigen: Hier wird eine Information über das aktuelle System ausgegeben (Hardware und Software). Sie dürfen sich hier auf die Intel-Architektur beschränken. Geben Sie also *ProcessorLevel* (Family) und *ProcessorRevision* (Model bzw. Stepping) nur aus, wenn es sich um Intel-CPUs handelt, ansonsten „unknown". Bei der Software beschränken Sie sich auf den Computernamen und die Betriebssystemversion, die ihre Applikation voraussetzt, wie auch ob Server-OS oder nicht. Eine Ausgabe kann etwa wie folgt aussehen:

```
System Information <HARDWARE>:
------------------------------
Processor Type                    : 586
Architecture                      : Intel CPU or compatibles
Processor Level (Family)          : 6
Revision                          : 0x2a07
Number of (logical) Processors    : 8

Features:
     Physical Address Extension not supported.
     Virtualization enabled.

System Information <SOFTWARE>:
------------------------------
Computer Name:   Zeus
OS Version   :   Windows 7 or greater
Server       :   No Server Version
```

Zur Realisierung:

Analyse: Analysieren Sie zuerst die Anforderungen und berücksichtigen Sie dabei, wo und wie ihnen die unten angeführten API-Calls helfen.

Design: Diskutieren Sie ihr Design und legen Sie die Datenstrukturen und Schnittstellen fest (Klassen & Methoden inkl. Parameter).

Implementierung: Implementieren Sie ihre Lösung mit Klassen (OOP).

Test: Erstellen Sie weiters einen Testtreiber, der diese Funktionen über ein einfaches Menü anbietet.

Dokumentation: Verfassen Sie eine Funktionsbeschreibung in LaTeX. Konzentrieren Sie sich dabei auf das Wesentliche.

Hinweise: Benützen Sie die Windows-Hilfe (Dokumentation zur Windows API), um sich mit den API-Calls und den Strukturen vertraut zu machen. Beginnen Sie bei folgenden API-Calls:

`GetCurrentProcess()`, `GetSystemInfo()`, `GetComputerName()`, `GetVersionEx()` bzw. die Version Helper Functions, `Process32First()`, `Process32Next()` bzw. Process Walking, `OpenProcess()`, `Set/GetPriorityClass()`, `TerminateProcess()`

Lassen Sie sich auch von dem jeweils angegebenen Beispiel-Code in der Hilfe inspirieren (etwa für die Prozessliste -> Bsp zu *Process Walking*), wo Sie die richtige Verwendung sehen.

Testen Sie ihr Programm ausführlich und geben Sie diese Tests auch ab!

Hinweise zur Ausarbeitung dieser Übung:

Lösen Sie diese Übung im Teamwork (2 Personen) und geben Sie eine gemeinsame Ausarbeitung ab. Setzen Sie dabei Bedarf an den entscheidenden Stellen **Pair Programming** ein, einer Technik aus dem Software Engineering Konzept *Extreme Programming* (XP von *Kent Beck*) und Scrum. Dabei sitzt das Teammitglied mit der geringeren Erfahrung am PC und codiert, während das zweite Teammitglied unterstützend und erklärend einwirkt und auch prüft, ob der geschriebene Code funktionieren kann (permanentes Review).

Sie können nach eigenem Ermessen im konventionellen Software Lifecycle arbeiten, oder weiter im XP- bzw. Scrum-Ansatz vorgehen.

- In XP/Scrum schreiben Sie **zuerst den Testtreiber** und werden sich dadurch bewusst, *welche Funktionalität* Sie benötigen und *in welcher Weise* Sie diese anwenden wollen.

- Danach beginnt das Design mit dem Fokus auf **Einfachheit** (*„the simplest thing that could possibly work“*).

- Die Implementierung wird in möglichst **kurzen Iterationen** immer ins Gesamtsystem integriert, um fehlende Übereinstimmungen möglichst früh zu erkennen. Sie können damit keine Funktionalitäten vergessen, weil Sie spätestens der nächste Testdurchlauf darauf hinweist.

- Da Testen zu hoher Produktstabilität führt, wird zwischendurch **immer wieder getestet** (*unit testing* durch SW-Entwickler und *functional testing* durch Kunden).

1 Beispiel 1 LKW:

2 Beispiel 2 Admin Tool:

2.1 Funktionsbeschreibung:

Das Admin Tool bietet folgende Funktionen:

- **OpenApp:** Öffnet eine Anwendung basierend auf dem übergebenen Pfad.
- **ListApps:** Listet alle laufenden Anwendungen auf.
- **TerminateApp:** Beendet eine Anwendung basierend auf der übergebenen PID.
- **ListSystemInfo:** Zeigt Informationen über das System an.

Die Funktionen sind mittels Erzeugen eines Command-Objekts ausführbar.

Im Testtreiber wurde ein einfaches Menü erstellt, welches in ähnlicher weise wie ein Konsolenmenü funktioniert.

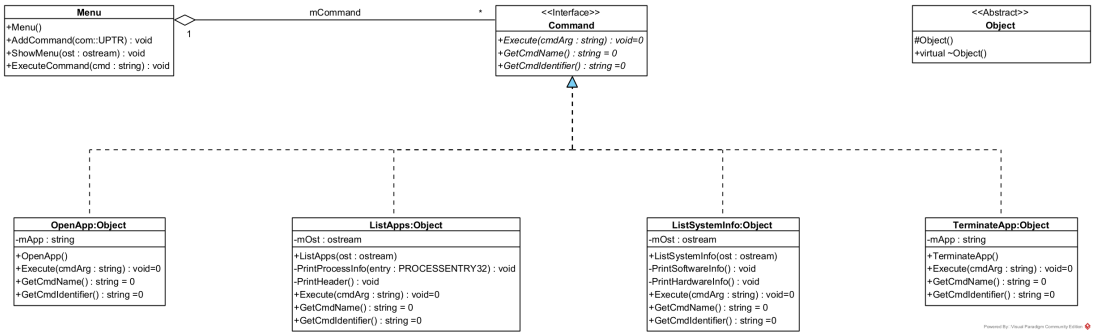
Mit folgenden Eingaben können die Funktionen getestet werden:

- **OpenApp:** "exec <Pfad zur Anwendung >"
- **ListApps:** "lsa"
- **TerminateApp:** "term <PID >"
- **ListSystemInfo:** "lss"
- **Beenden:** "exit"

Die Anwendung wird nur korrekt beendet, wenn "exit" eingegeben wird. Ansonsten treten Memory Leaks auf, da die Command-Objekte nicht korrekt gelöscht werden.

2.2 Design:

Für diese Aufgabenstellung wurde das Command Pattern angewandt. Dazu wird ein Interface Command als Abstraktion für die verschiedenen Kommandos erstellt. Die konkreten Kommandos (OpenApp, ListApps, TerminateApp, ListSystemInfo) implementieren dieses Interface und enthalten die jeweilige Logik für die Ausführung der Kommandos. Die Menu-Klasse dient als Invoker, der die Kommandos verwaltet und ausführt.



2.2.1 Object.h

```
#ifndef OBJECT_HPP
#define OBJECT_HPP

/**
 * @brief Base Object class.
 */

class Object
{
public:
    virtual ~Object() = default;

protected:
    Object() = default;
};

#endif
```

2.2.2 Command.h

```cpp
#ifndef COMMAND_HPP
#define COMMAND_HPP

#include <string>
#include <memory>
#include <vector>

/**
 * @brief Base interface for executable commands in the AdminTool.
 */
class Command
{
public:

        using UPTR = std::unique_ptr<Command>;

        using TCont = std::vector<Command::UPTR>;

        /**
         * @brief Executes the command with the provided argument.
         * @param cmdArg Argument string required for command execution.
         */
        virtual void Execute(const std::string& cmdArg) = 0;

        /**
         * @brief Gets the full name and usage description of the command.
         * @return A string containing the command name and usage.
         */
        virtual std::string GetCmdName() const = 0;

        /**
         * @brief Gets the raw identifier of the command used for invocation.
         * @return The command identifier string.
         */
        virtual std::string GetCmdIdentifier() const = 0;

        virtual ~Command() = default;
};

#endif
```

2.2.3 OpenApp.h

```cpp
#ifndef OPENAPP_HPP
#define OPENAPP_HPP

#include "Command.hpp"
#include "Object.hpp"

#include <string>

#include <iostream>

/**
 * @brief Command to open an external application.
 */
class OpenApp : public Command, public Object
{
public:

        inline static constexpr const char* ERROR_CREATE_PROCESS_FAILED = "Failed␣to␣create␣process.␣Error:␣";


        /**
         * @brief Default constructor.
         */
        OpenApp() = default;

        /**
         * @brief Executes the open application command.
         * @param cmdArg The application name or path to execute.
         */
        virtual void Execute(const std::string& cmdArg) override;

        /**
         * @brief Gets the command name and usage.
         * @return Command name and usage string.
         */
        virtual std::string GetCmdName() const override;

        /**
         * @brief Gets the command identifier.
         * @return The identifier string.
         */
        virtual std::string GetCmdIdentifier() const override;


private:

        inline static constexpr const char* CMD_IDENTIFIER = "exec";

};

#endif
```

2.2.4 OpenApp.cpp

```cpp
/*****************************************************************//**
 /**
 * \file   OpenApp.cpp
 * \brief  Implementation of the OpenApp command.
 *
 * \author Simon
 * \date   March 2026
 ********************************************************************/

#include "OpenApp.hpp"
#include "windows.h"
#include "Hlp.h"
#include <iostream>
#include <vector>


void OpenApp::Execute(const std::string& cmdArg)
{
    if (cmdArg.empty())
        {
                throw std::invalid_argument("No application specified.");
        }

        // parameters for creating a new process
        //ost << "Enter the name of the application to open (e.g. mspaint): " << std::endl << ">";
        //std::string app;
        //in >> app;

        STARTUPINFO si; // specifies how the main window should appear [in]
        PROCESS_INFORMATION pi; // filled by CreateProcess with information about
        // the newly created process and its primary thread [out]

        memset(&si, 0, sizeof(si)); // set all parameters of si (struct) to zero
        // memset sets every byte to the specific value -> not the specific datatypes!!!

        si.cb = sizeof(si); // set the size of the startupinfo (byte count)

        memset(&pi, 0, sizeof(pi));

        // CreateProcess may modify the command line buffer; provide a writable copy.
        std::vector<char> cmdLine(cmdArg.begin(), cmdArg.end());
        cmdLine.push_back('\0');

        // start the new process
        if (!CreateProcess(
                0,                  // no module name, use command line
     cmdLine.data(),  // command line
                0,                  // no process security attribute
                0,                  // no thread security attribute
                false,      // no inheritance
                0,                  // no creation flags
                0,                      // use parent's environment block
                0,                      // use parent's starting directory
                &si,            // pointer to the startupinfo struct
                &pi                     // pointer to the process_information struct
        )) {
                throw std::runtime_error(ERROR_CREATE_PROCESS_FAILED + Hlp::ErrMsg(GetLastError()));
        }

        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
}

std::string OpenApp::GetCmdName() const
{
        return std::string(OpenApp::CMD_IDENTIFIER) + " -> Open Application (Usage: exec <app_name>)";
}

std::string OpenApp::GetCmdIdentifier() const
{
        return OpenApp::CMD_IDENTIFIER;
}
```

2.2.5 ListApps.hpp

```cpp
#ifndef LISTAPPS_HPP
#define LISTAPPS_HPP

#include "Command.hpp"
#include "Object.hpp"
#include "windows.h"
#include "tlhelp32.h"
#include <iostream>

/**
 * @brief Command to list running applications and their process information.
 */
class ListApps : public Command, public Object
{
public:

        inline static constexpr const char* ERROR_SNAPSHOT_FAILED = "Failed␣to␣create␣snapshot␣of␣processes.␣Error:␣";
        inline static constexpr const char* ERROR_BAD_OSTREAM = "Output␣stream␣is␣in␣a␣bad␣state.";

        inline static constexpr size_t cLowPrioThreshold = 4;
        inline static constexpr size_t cHighPrioThreshold = 10;
        /**
         * @brief Constructor taking an output stream for printing.
         * @param ost The output stream to print to.
         */
        explicit ListApps(std::ostream& ost) : mOst{ ost } {}

        /**
         * @brief Executes the process listing command.
         * @param cmdArg Command arguments (ignored).
         */
        virtual void Execute(const std::string& cmdArg) override;

        /**
         * @brief Gets the command name and usage.
         * @return Command name and usage string.
         */
        virtual std::string GetCmdName() const override;

        /**
         * @brief Gets the command identifier.
         * @return The identifier string.
         */
        virtual std::string GetCmdIdentifier() const override;


private:

        /**
         * @brief Prints detailed information about a single process.
         * @param entry The process entry to print.
         */
        void PrintProcessInfo(const PROCESSENTRY32 & entry) const;

        /**
         * @brief Prints the header for the process list.
         */
        void PrintHeader() const;

        std::ostream & mOst;

        inline static constexpr size_t cProcessNameWidth = 40;
        inline static constexpr size_t cProcessPrioWidth = 15;
        inline static constexpr size_t cProcessPIDWidth = 10;
        inline static constexpr size_t cProcessCntTHWidth = 15;


        inline static constexpr const char* CMD_IDENTIFIER = "lsa";
};

#endif
```

2.2.6 ListApps.cpp

```cpp
#include <iomanip>
/**
 * @file ListApps.cpp
 * @brief Implementation of the ListApps command.
 */

#include "ListApps.hpp"
#include "windows.h"
#include <iostream>


void ListApps::Execute(const std::string& cmdArg)
{
        HANDLE hProcessSnap;
        PROCESSENTRY32 entry;

        hProcessSnap = CreateToolhelp32Snapshot(TH32CS_SNAPALL,NULL);

        if(hProcessSnap == INVALID_HANDLE_VALUE)
        {
                throw  std::invalid_argument( ERROR_SNAPSHOT_FAILED + GetLastError());
        }

        PrintHeader();

        bool retval = Process32First(hProcessSnap,&entry);

        while (retval) {
                PrintProcessInfo(entry);
                retval = Process32Next(hProcessSnap,&entry);
        }

        CloseHandle(hProcessSnap);
}

std::string ListApps::GetCmdName() const
{
        return std::string(ListApps::CMD_IDENTIFIER) + " -> List Applications (Usage: lsa)";
}

std::string ListApps::GetCmdIdentifier() const
{
        return ListApps::CMD_IDENTIFIER;
}


static const char* PriorityToString(const LONG priority)
{
        if (priority <= ListApps::cLowPrioThreshold)  return "low";
        if (priority >= ListApps::cHighPrioThreshold) return "high";
        return "normal";
}

void ListApps::PrintHeader() const
{
        if (mOst.bad()) {
                throw std::runtime_error(ERROR_BAD_OSTREAM);
        }

        mOst << std::left
                << std::setw(cProcessNameWidth) << "Process"
                << std::setw(cProcessPIDWidth) << "PID"
                << std::setw(cProcessPrioWidth) << "BasePriority"
                << std::setw(cProcessCntTHWidth) << "Thread Count"
                << '\n';

        mOst << std::string(cProcessNameWidth+ cProcessPIDWidth+ cProcessPrioWidth+ cProcessCntTHWidth, '-') << '\n';
}

void ListApps::PrintProcessInfo(const PROCESSENTRY32& entry) const
{
        if (mOst.bad()) {
                throw std::runtime_error(ERROR_BAD_OSTREAM);
        }

        mOst << std::left
                << std::setw(cProcessNameWidth) << entry.szExeFile
                << std::setw(cProcessPIDWidth) << entry.th32ProcessID
                << std::setw(cProcessPrioWidth) << PriorityToString(entry.pcPriClassBase)
                << std::setw(cProcessCntTHWidth) << entry.cntThreads
                << '\n';
}
```

2.2.7 TerminateApp.hpp

```cpp
#ifndef TERMINATEAPP_HPP
#define TERMINATEAPP_HPP

#include "Command.hpp"
#include "Object.hpp"
#include "windows.h"
#include <iostream>

/**
 * @brief Command to terminate an application by process ID.
 */
class TerminateApp : public Command, public Object
{
public:

        inline static constexpr const char* ERROR_TERMINATE_PROCESS_FAILED = "Failed to terminate process. Error: ";

        /**
         * @brief Default constructor.
         */
        TerminateApp() = default;

        /**
         * @brief Executes the process termination command.
         * @param cmdArg The process ID as a string.
         */
        virtual void Execute(const std::string& cmdArg) override;

        /**
         * @brief Gets the command name and usage.
         * @return Command name and usage string.
         */
        virtual std::string GetCmdName() const override;

        /**
         * @brief Gets the command identifier.
         * @return The identifier string.
         */
        virtual std::string GetCmdIdentifier() const override;


private:

        inline static constexpr const char* CMD_IDENTIFIER = "term";
};

#endif
```

2.2.8 ListSystemInfo.cpp

```cpp

/**
 * @file ListSystemInfo.cpp
 * @brief Implementation of the ListSystemInfo command.
 */
#include "ListSystemInfo.hpp"
#include <windows.h>
#include <VersionHelpers.h>
#include <iostream>
#include <iomanip>
#include <string>

static const char* GetArchitectureString(WORD arch)
{
    switch (arch)
    {
    case PROCESSOR_ARCHITECTURE_INTEL:
        return "Intel CPU or compatibles";
    case PROCESSOR_ARCHITECTURE_AMD64:
        return "x64 (AMD or Intel)";
    case PROCESSOR_ARCHITECTURE_ARM:
        return "ARM";
    case PROCESSOR_ARCHITECTURE_ARM64:
        return "ARM64";
    default:
        return "unknown";
    }
}

static std::string GetOSVersionString()
{
    if (IsWindows10OrGreater())
        return "Windows 10 or greater";
    if (IsWindows8Point1OrGreater())
        return "Windows 8.1 or greater";
    if (IsWindows8OrGreater())
        return "Windows 8 or greater";
    if (IsWindows7OrGreater())
        return "Windows 7 or greater";

    return "Older than Windows 7";
}


void ListSystemInfo::Execute(const std::string& cmdArg)
{
        PrintHardwareInfo();
        PrintSoftwareInfo();
}

void ListSystemInfo::PrintHardwareInfo()
{
    if(mOst.bad())
    {
        throw std::runtime_error(ERROR_BAD_OSTREAM);
        }

    SYSTEM_INFO si{};
    GetSystemInfo(&si);

    mOst << "System Information <HARDWARE>:\n";
    mOst << "------------------------------\n";

    mOst << "Processor Type : " << si.dwProcessorType << "\n";
    mOst << "Architecture : " << GetArchitectureString(si.wProcessorArchitecture) << "\n";

    if (si.wProcessorArchitecture == PROCESSOR_ARCHITECTURE_INTEL)
    {
        mOst << "Processor Level (Family) : " << si.wProcessorLevel << "\n";
        mOst << "Revision : 0x"
            << std::hex << std::setw(4) << std::setfill('0')
            << si.wProcessorRevision
            << std::dec << std::setfill(' ') << "\n";
    }
    else
    {
        mOst << "Processor Level (Family) : unknown\n";
        mOst << "Revision : unknown\n";
    }

    mOst << "Number of (logical) Processors : " << si.dwNumberOfProcessors << "\n";
    mOst << "Features:\n";

    if (IsProcessorFeaturePresent(PF_PAE_ENABLED))
        mOst << "Physical Address Extension supported.\n";
    else
        mOst << "Physical Address Extension not supported.\n";

    if (IsProcessorFeaturePresent(PF_VIRT_FIRMWARE_ENABLED))
        mOst << "Virtualization enabled.\n";
    else
        mOst << "Virtualization not enabled.\n";
}

void ListSystemInfo::PrintSoftwareInfo()
{
    if (mOst.bad())
    {
        throw std::runtime_error(ERROR_BAD_OSTREAM);
    }

    char computerName[MAX_COMPUTERNAME_LENGTH + 1]{};
    DWORD size = MAX_COMPUTERNAME_LENGTH + 1;

```

```
    if (!GetComputerNameA(computerName, &size))
    {
        lstrcpyA(computerName, "unknown");
    }
    mOst << std::endl;
    mOst << "System Information <SOFTWARE>:\n";
    mOst << "-----------------------------\n";
    mOst << "Computer Name: " << computerName << "\n";
    mOst << "OS Version : " << GetOSVersionString() << "\n";
    mOst << "Server : " << (IsWindowsServer() ? "Server Version" : "No Server Version") << "\n";
}



std::string ListSystemInfo::GetCmdName() const
{
        return std::string(ListSystemInfo::CMD_IDENTIFIER) + " -> List System Information (Usage: lss)";
}

std::string ListSystemInfo::GetCmdIdentifier() const
{
        return ListSystemInfo::CMD_IDENTIFIER;
}
```

2.2.9 ListSystemInfo.hpp

```cpp
#ifndef LISTSYSTEMINFO_HPP
#define LISTSYSTEMINFO_HPP

#include "Command.hpp"
#include "Object.hpp"
#include <windows.h>
#include <iostream>

/**
 * @brief Command to list system hardware and software information.
 */
class ListSystemInfo : public Command, public Object
{
public:

        inline static constexpr const char* ERROR_BAD_OSTREAM = "Output stream is in a bad state.";

        /**
         * @brief Constructor taking an output stream for printing.
         * @param ost The output stream to print to.
         */
        explicit ListSystemInfo(std::ostream & ost) : mOst{ ost } {}

        /**
         * @brief Executes the system info listing command.
         * @param cmdArg Command arguments (ignored).
         */
        virtual void Execute(const std::string& cmdArg) override;

        /**
         * @brief Gets the command name and usage.
         * @return Command name and usage string.
         */
        virtual std::string GetCmdName() const override;

        /**
         * @brief Gets the command identifier.
         * @return The identifier string.
         */
        virtual std::string GetCmdIdentifier() const override;

private:

        /**
         * @brief Prints software information of the system.
         */
        void PrintSoftwareInfo();

        /**
         * @brief Prints hardware information of the system.
         */
        void PrintHardwareInfo();

        std::ostream& mOst;

        inline static constexpr const char* CMD_IDENTIFIER = "lss";
};

#endif
```

2.2.10 TerminateApp.cpp

```cpp
/**
 * @file TerminateApp.cpp
 * @brief Implementation of the TerminateApp command.
 */
#include "TerminateApp.hpp"
#include <sstream>
#include <string>
#include "Hlp.h"



void TerminateApp::Execute(const std::string & cmdArg)
{

        DWORD mApp = 0;

        std::stringstream converter;
        converter << cmdArg;
        converter >> mApp;

        HANDLE hProcess = OpenProcess(PROCESS_TERMINATE, FALSE, mApp);

        if(hProcess == NULL)
        {
                throw std::runtime_error(ERROR_TERMINATE_PROCESS_FAILED + std::to_string(mApp) + ". Error: " + Hlp::ErrMsg(
                     GetLastError()));
        }

        TerminateProcess(hProcess, 0);
   CloseHandle(hProcess);
}

std::string TerminateApp::GetCmdName() const
{

        std::stringstream converter;

        converter << TerminateApp::CMD_IDENTIFIER;
        converter << " -> Terminate Application (Usage: term <PID>)";

        return converter.str();
}

std::string TerminateApp::GetCmdIdentifier() const
{
        return TerminateApp::CMD_IDENTIFIER;
}
```

2.2.11 Menu.hpp

```cpp
#ifndef MENU_HPP
#define MENU_HPP

#include "Object.hpp"
#include "Command.hpp"
#include <iostream>


/**
 * @brief Manages a collection of commands and executes them.
 */
class Menu : public Object
{
public :

        inline static constexpr const char* ERROR_BAD_OSTREAM = "Error␣Bad␣Ostream␣Provided!";

        /**
         * @brief Default constructor.
         */
        Menu() = default;

        /**
         * @brief Adds a new command to the menu.
         * @param com Unique pointer to the command to add.
         */
        void AddCommand(Command::UPTR com);

        /**
         * @brief Displays the available menu commands to the given output stream.
         * @param ost Output stream for display target.
         */
        void ShowMenu(std::ostream & ost);

        /**
         * @brief Executes a command by its identifier phrase.
         * @param cmd The command line string (identifier + arguments).
         */
        void ExecuteCommand(const std::string& cmd);

private:

        Command::TCont mCommands;
};

#endif // !MENU_HPP
```

2.2.12 Menu.cpp

```cpp
#include "Menu.hpp"
/**
 * @file Menu.cpp
 * @brief Implementation of the Menu class.
 */
#include "Menu.hpp"
#include <cassert>
#include <algorithm>
#include <sstream>

static std::ostream & hline(std::ostream & os)
{
        assert(os.good());

        return os << "----------------------------------------" << std::endl;
}



void Menu::AddCommand(Command::UPTR com)
{
        mCommands.push_back(std::move(com));
}

void Menu::ShowMenu(std::ostream& ost)
{
        if(ost.fail())
        {
                throw std::runtime_error(ERROR_BAD_OSTREAM);
        }

        ost << hline;
        ost << "Available Commands: " << std::endl;

        size_t num = 0;

        std::for_each(mCommands.cbegin(),mCommands.cend(),[&](const auto & cmd)
        {
                ost << num++ << " " << cmd->GetCmdName() << std::endl;
        });
}

void Menu::ExecuteCommand(const std::string & cmd)
{

        std::stringstream converter;

        converter << cmd;

        std::string cmdIdentifier;
        // extract the first word of the command as identifier
        converter >> cmdIdentifier;

        const auto & find = std::find_if(mCommands.cbegin(), mCommands.cend(), [&](const Command::UPTR & com) {
                return com->GetCmdIdentifier() == cmdIdentifier;
        });

        try {

                if (find != mCommands.cend())
                {
                        std::string cmdArg;
                        // extract the rest of the command as argument
                        std::getline(converter >> std::ws, cmdArg); // read the rest of the line without leading whitespace
                        (*find)->Execute(cmdArg);
                }
                else
                {
                        std::cerr << "Unknown Command: " << cmd << std::endl;
                }
        }
        catch (const std::exception & ex)
        {
                std::cerr << "Error executing command: " << ex.what() << std::endl;
        }
        catch (...) {
                std::cerr << "Unknown error executing command: " << cmd << std::endl;
        }
}
```

2.2.13 OpenApp.cpp

```cpp
/*****************************************************************//**
 /**
 * \file   OpenApp.cpp
 * \brief  Implementation of the OpenApp command.
 *
 * \author Simon
 * \date   March 2026
 ********************************************************************/

#include "OpenApp.hpp"
#include "windows.h"
#include "Hlp.h"
#include <iostream>
#include <vector>


void OpenApp::Execute(const std::string& cmdArg)
{
    if (cmdArg.empty())
        {
                throw std::invalid_argument("No application specified.");
        }

        // parameters for creating a new process
        //ost << "Enter the name of the application to open (e.g. mspaint): " << std::endl << ">";
        //std::string app;
        //in >> app;

        STARTUPINFO si; // specifies how the main window should appear [in]
        PROCESS_INFORMATION pi; // filled by CreateProcess with information about
        // the newly created process and its primary thread [out]

        memset(&si, 0, sizeof(si)); // set all parameters of si (struct) to zero
        // memset sets every byte to the specific value -> not the specific datatypes!!!

        si.cb = sizeof(si); // set the size of the startupinfo (byte count)

        memset(&pi, 0, sizeof(pi));

        // CreateProcess may modify the command line buffer; provide a writable copy.
        std::vector<char> cmdLine(cmdArg.begin(), cmdArg.end());
        cmdLine.push_back('\0');

        // start the new process
        if (!CreateProcess(
                0,                  // no module name, use command line
     cmdLine.data(),  // command line
                0,                  // no process security attribute
                0,                  // no thread security attribute
                false,      // no inheritance
                0,                  // no creation flags
                0,                      // use parent's environment block
                0,                      // use parent's starting directory
                &si,            // pointer to the startupinfo struct
                &pi                     // pointer to the process_information struct
        )) {
                throw std::runtime_error(ERROR_CREATE_PROCESS_FAILED + Hlp::ErrMsg(GetLastError()));
        }

        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
}

std::string OpenApp::GetCmdName() const
{
        return std::string(OpenApp::CMD_IDENTIFIER) + " -> Open Application (Usage: exec <app_name>)";
}

std::string OpenApp::GetCmdIdentifier() const
{
        return OpenApp::CMD_IDENTIFIER;
}
```

2.2.14 Testausgabe:

```
Test Cmd Identifier
[Test OK] Result: (Expected: lsa == Result: lsa)

--------------------------
Executing ListApps Command...
Process                                    PID        BasePriority    Thread
   ↪ Count
-------------------------------------------------------------------------------
   ↪
[System Process]                           0          low             60
System                                     4          normal          518
Secure System                              284        normal          0
Registry                                   328        normal          4
smss.exe                                   940        high            2
csrss.exe                                  1656       high            15
wininit.exe                                1864       high            2
services.exe                               2000       normal          8
LsaIso.exe                                 2036       normal          3
lsass.exe                                  2044       normal          12
svchost.exe                                1340       normal          20
fontdrvhost.exe                            1648       normal          5
svchost.exe                                2148       normal          12
WUDFCompanionHost.exe                      2160       high            4
svchost.exe                                2424       normal          19
svchost.exe                                2432       normal          92
svchost.exe                                2444       high            60
amdpmfservice.exe                          2464       normal          10
svchost.exe                                2480       normal          19
svchost.exe                                2488       high            40
svchost.exe                                2500       normal          10
svchost.exe                                2732       normal          12
svchost.exe                                2864       normal          4
svchost.exe                                2892       normal          13
svchost.exe                                3112       normal          5
svchost.exe                                3360       normal          8
vmms.exe                                   3784       normal          12
svchost.exe                                3872       normal          13
WUDFHost.exe                               3880       high            5
amdfendrsr.exe                             3952       normal          4
atiesrxx.exe                               3960       normal          48
svchost.exe                                4028       normal          17
svchost.exe                                3080       normal          6
vmcompute.exe                              4372       normal          2
WUDFHost.exe                               4448       normal          5
dasHost.exe                                4536       normal          3
svchost.exe                                4560       normal          6
svchost.exe                                4796       normal          15
WUDFCompanionHost.exe                      4860       high            3
WUDFCompanionHost.exe                      4948       high            3
WmiPrvSE.exe                               5076       normal          6
WUDFHost.exe                               4336       normal          7
svchost.exe                                4320       normal          6
Memory Compression                         4324       normal          70
svchost.exe                                5172       normal          8
```

```
svchost.exe                                5204      normal          8
svchost.exe                                5332      normal          4
svchost.exe                                5404      normal          16
spoolsv.exe                                5468      normal          9
WUDFHost.exe                               5504      normal          16
svchost.exe                                5872      normal          6
WUDFHost.exe                               6048      normal          5
AmdPpkgSvc.exe                             6124      normal          2
svchost.exe                                4240      normal          9
service.exe                                5124      normal          3
DAX3API.exe                                5720      normal          477
mDNSResponder.exe                          5724      normal          2
svchost.exe                                5748      normal          9
GlideXNearService.exe                      5764      normal          11
Lenovo.Modern.ImController.exe             6148      normal          22
GlideXServiceExt.exe                       6164      normal          4
LenovoUtilityService.exe                   6172      normal          6
ElevocControlService.exe                   6180      normal          9
jtagserver.exe                             6200      normal          4
GlideXRemoteService.exe                    6208      normal          5
GameInputRedistService.exe                 6236      normal          10
FoxitPDFReaderUpdateService.exe            6248      normal          3
GlideXService.exe                          6276      normal          17
RtkAudUService64.exe                       6388      normal          17
LNBITSSvc.exe                              6396      normal          9
LenovoVantageService.exe                   6416      low             18
ReadyForService.exe                        6540      normal          3
SmartSense.exe                             6584      normal          4
UDClientService.exe                        6688      normal          42
sqlwriter.exe                              6696      normal          2
vmware-authd.exe                           6708      normal          4
vmware-usbarbitrator64.exe                 6716      normal          3
vmnetdhcp.exe                              6728      normal          2
vmnat.exe                                  6744      normal          4
wireguard.exe                              6796      normal          23
wslservice.exe                             6864      normal          3
svchost.exe                                6888      normal          4
NgcIso.exe                                 6976      high            1
WUDFHost.exe                               6408      normal          5
WmiPrvSE.exe                               8768      normal          9
unsecapp.exe                               10100     normal          4
wlanext.exe                                6672      normal          1
svchost.exe                                11068     normal          7
svchost.exe                                12208     normal          3
svchost.exe                                12272     normal          11
MidiSrv.exe                                11576     normal          3
WorkloadsSessionManager.exe                15904     normal          2
LenovoVantage-(VantageCoreAddin).exe       4460      low             17
SecurityHealthService.exe                  19820     normal          11
svchost.exe                                3316      normal          8
Locator.exe                                3264      normal          2
WmiApSrv.exe                               20396     normal          2
AppProvisioningPlugin.exe                  10088     normal          10
conhost.exe                                2032      normal          4
svchost.exe                                26440     normal          19
```

```
svchost.exe                          24328     normal        9
adb.exe                              12352     normal        5
WorkloadsSessionHost.exe             22620     normal        43
spacedeskService.exe                 35220     normal        7
WUDFHost.exe                         5440      normal        6
svchost.exe                          40936     normal        3
logioptionsplus_updater.exe          15136     normal        41
FileSyncHelper.exe                   33888     normal        6
WaaSMedicAgent.exe                   24640     normal        8
conhost.exe                          49040     normal        2
gamingservices.exe                   40544     normal        18
gamingservicesnet.exe                37592     normal        39
mysqld.exe                           18672     normal        7
OfficeClickToRun.exe                 28928     normal        19
SearchIndexer.exe                    39316     normal        11
svchost.exe                          840       normal        28
MpDefenderCoreService.exe            45292     normal        8
MsMpEng.exe                          40308     normal        79
NisSrv.exe                           48628     normal        5
audiodg.exe                          12584     normal        14
csrss.exe                            44880     high          18
winlogon.exe                         51408     high          3
fontdrvhost.exe                      32160     normal        5
dwm.exe                              22728     high          72
atieclxx.exe                         23816     normal        16
amdpmfserviceuser.exe                19268     normal        7
AutoModeDetect.exe                   14368     normal        12
wireguard.exe                        12424     normal        23
conhost.exe                          50844     normal        3
conhost.exe                          23972     normal        2
FnHotkeyCapsLKNumLK.exe              21848     normal        2
dllhost.exe                          12240     normal        7
spacedeskServiceTray.exe             23656     normal        3
unsecapp.exe                         12448     normal        3
FnHotkeyUtility.exe                  22896     normal        14
logioptionsplus_agent.exe            38532     normal        131
sihost.exe                           48828     normal        13
svchost.exe                          7328      normal        2
svchost.exe                          14268     normal        12
svchost.exe                          25992     normal        4
taskhostw.exe                        26784     normal        6
unsecapp.exe                         51580     normal        3
taskhostw.exe                        49812     normal        4
explorer.exe                         20324     normal        236
ShellHost.exe                        23308     normal        26
CrossDeviceResume.exe                47356     normal        25
PowerToys.exe                        49004     normal        15
svchost.exe                          46088     normal        7
SearchHost.exe                       20724     normal        32
StartMenuExperienceHost.exe          20372     normal        35
WidgetBoard.exe                      26500     normal        54
RuntimeBroker.exe                    51456     normal        12
svchost.exe                          41072     normal        3
WidgetService.exe                    8512      normal        7
msedgewebview2.exe                   20368     normal        48
```

```
msedgewebview2.exe                    18496    normal    7
msedgewebview2.exe                    27804    high      40
msedgewebview2.exe                    16704    normal    29
msedgewebview2.exe                    42624    normal    9
msedgewebview2.exe                    46856    normal    32
backgroundTaskHost.exe                40424    normal    18
MicrosoftStartFeedProvider.exe        43000    normal    10
SmartSenseController.exe              27244    normal    3
conhost.exe                           52372    normal    3
PowerToys.AlwaysOnTop.exe             43820    normal    3
PowerToys.Awake.exe                   20204    normal    14
ctfmon.exe                            39264    high      12
Microsoft.CmdPal.UI.exe               47544    normal    61
PowerToys.ColorPickerUI.exe           40948    normal    15
PowerToys.CropAndLock.exe             16012    normal    3
WindowsPackageManagerServer.exe       36256    normal    6
PowerToys.FancyZones.exe              38436    normal    45
PowerToys.KeyboardManagerEngine.exe   22804    high      4
PowerToys.Peek.UI.exe                 52720    normal    33
TextInputHost.exe                     50300    high      42
PowerToys.PowerLauncher.exe           48152    normal    20
RuntimeBroker.exe                     31504    normal    3
PhoneExperienceHost.exe               32140    normal    40
SecurityHealthSystray.exe             12792    normal    1
LockApp.exe                           21884    normal    33
RuntimeBroker.exe                     9472     normal    36
svchost.exe                           43240    high      27
RtkAudUService64.exe                  37576    normal    8
svchost.exe                           28916    normal    7
logioptionsplus_appbroker.exe         28876    normal    9
WUDFHost.exe                          37844    normal    6
UserSSCtrl.exe                        11156    normal    3
OneDrive.exe                          47672    normal    75
OneDrive.exe                          19932    normal    70
msedgewebview2.exe                    51872    normal    52
msedgewebview2.exe                    36896    normal    7
msedgewebview2.exe                    6104     high      42
msedgewebview2.exe                    14280    normal    27
msedgewebview2.exe                    30388    normal    9
msedgewebview2.exe                    4728     normal    15
msedgewebview2.exe                    24744    low       32
OneDrive.Sync.Service.exe             33744    normal    19
ms-teams.exe                          29116    normal    81
MathWorksServiceHost.exe              18424    normal    101
RuntimeBroker.exe                     38724    normal    2
msedgewebview2.exe                    5352     normal    52
msedgewebview2.exe                    11396    normal    8
msedgewebview2.exe                    32508    high      102
msedgewebview2.exe                    504      normal    21
msedgewebview2.exe                    25020    normal    10
msedgewebview2.exe                    532      normal    37
ms-teams.exe                          20640    normal    109
MathWorksServiceHost-Monitor.exe      39368    normal    27
msedgewebview2.exe                    49848    normal    10
Discord.exe                           48548    normal    52
```

```
Discord.exe                                21196     normal          7
Discord.exe                                33524     high            159
Discord.exe                                13840     normal          20
Discord.exe                                2172      normal          96
Discord.exe                                15908     normal          7
AMDRSServ.exe                              43108     normal          11
AMDRSSrcExt.exe                            41132     normal          30
amdow.exe                                  3744      normal          24
WhatsApp.Root.exe                          30012     low             72
RuntimeBroker.exe                          47840     normal          2
msedgewebview2.exe                         22760     normal          47
msedgewebview2.exe                         36220     normal          7
msedgewebview2.exe                         684       high            18
msedgewebview2.exe                         20232     normal          23
msedgewebview2.exe                         47344     normal          9
msedgewebview2.exe                         41760     low             39
AppleMobileDeviceProcess.exe               18804     normal          9
ONENOTEM.EXE                               30128     normal          2
acrotray.exe                               15232     normal          1
jusched.exe                                868       normal          3
firefox.exe                                10256     normal          78
crashhelper.exe                            16308     normal          2
firefox.exe                                38652     high            86
firefox.exe                                32728     normal          6
firefox.exe                                46384     low             27
firefox.exe                                18448     normal          26
firefox.exe                                38084     normal          27
firefox.exe                                46896     normal          5
olk.exe                                    6272      normal          36
msedgewebview2.exe                         51868     normal          49
msedgewebview2.exe                         2328      normal          7
msedgewebview2.exe                         17760     high            41
msedgewebview2.exe                         49764     normal          21
msedgewebview2.exe                         18936     normal          10
msedgewebview2.exe                         48324     normal          33
msedgewebview2.exe                         20360     normal          37
RuntimeBroker.exe                          12408     normal          3
olkexthost.exe                             21776     normal          5
RuntimeBroker.exe                          41172     normal          3
LenovoVantage-(GenericMessagingAddin).exe2092        low             14
backgroundTaskHost.exe                     17196     normal          6
RuntimeBroker.exe                          28812     normal          2
FileCoAuth.exe                             8636      normal          7
firefox.exe                                16544     low             27
AppActions.exe                             50092     normal          7
jucheck.exe                                15084     normal          4
SystemSettings.exe                         33732     normal          63
ApplicationFrameHost.exe                   17064     normal          24
svchost.exe                                33352     normal          3
UserOOBEBroker.exe                         36856     normal          1
GitHubDesktop.exe                          50700     normal          40
GitHubDesktop.exe                          42864     high            38
GitHubDesktop.exe                          6884      normal          21
GitHubDesktop.exe                          28192     low             38
firefox.exe                                25520     normal          5
```

```
firefox.exe                                49336     low          29
firefox.exe                                28304     low          28
firefox.exe                                22784     normal       5
RuntimeBroker.exe                          31256     normal       2
firefox.exe                                3320      low          29
Spotify.exe                                11308     normal       84
Spotify.exe                                42372     normal       6
Spotify.exe                                52764     high         101
Spotify.exe                                44384     normal       30
Spotify.exe                                672       normal       11
backgroundTaskHost.exe                     11400     normal       4
RuntimeBroker.exe                          46016     normal       2
Spotify.exe                                776       low          35
Spotify.exe                                15676     normal       10
firefox.exe                                3188      normal       28
firefox.exe                                24732     normal       26
firefox.exe                                12828     normal       26
svchost.exe                                44140     normal       4
vctip.exe                                  45384     normal       21
Visual Paradigm.exe                        22324     normal       50
devenv.exe                                 22612     normal       125
PerfWatson2.exe                            45848     normal       16
StandardCollector.Service.exe              35044     normal       24
Microsoft.ServiceHub.Controller.exe        7568      normal       18
ServiceHub.VSDetouredHost.exe              41964     normal       21
ServiceHub.ThreadedWaitDialog.exe          43800     normal       15
ServiceHub.IdentityHost.exe                43700     normal       26
vshost.exe                                 33228     normal       1
ServiceHub.IntellicodeModelService.exe     11392     normal       8
ServiceHub.IndexingService.exe             15020     normal       19
ServiceHub.Host.dotnet.x64.exe             18704     normal       24
copilot-language-server.exe                49436     normal       15
conhost.exe                                49916     normal       2
ServiceHub.TestWindowStoreHost.exe         52712     normal       15
svchost.exe                                53752     normal       6
MSBuild.exe                                10684     normal       15
conhost.exe                                48240     normal       2
ServiceHub.DataWarehouseHost.exe           3972      normal       22
firefox.exe                                16956     normal       31
ServiceHub.Host.dotnet.x64.exe             51792     normal       26
OpenConsole.exe                            39152     normal       5
powershell.exe                             39272     normal       13
vcpkgsrv.exe                               29084     normal       6
vcpkgsrv.exe                               45708     normal       9
VsDebugConsole.exe                         46460     normal       5
conhost.exe                                32932     normal       6
OpenConsole.exe                            18056     normal       11
WindowsTerminal.exe                        5832      normal       56
AdminTool.exe                              37356     normal       7
msvsmon.exe                                30864     normal       12
msedgewebview2.exe                         31220     normal       63
msedgewebview2.exe                         32776     normal       10
msedgewebview2.exe                         51504     high         38
msedgewebview2.exe                         44356     normal       28
msedgewebview2.exe                         51032     normal       9
```

```
msedgewebview2.exe                          44820     normal          19
msedgewebview2.exe                          53300     normal          18
vcpkgsrv.exe                                53272     normal          10
--------------------------
Test Exception in TestCase
[Test OK] Result: (Expected: true == Result: true)

Test Exception Error Bad Ost
[Test OK] Result: (Expected: 0 == Result: 0)

Test Cmd Identifier
[Test OK] Result: (Expected: lss == Result: lss)

--------------------------
Executing ListSysInfo Command...
System Information <HARDWARE>:
------------------------------
Processor Type : 8664
Architecture : x64 (AMD or Intel)
Processor Level (Family) : unknown
Revision : unknown
Number of (logical) Processors : 20
Features:
Physical Address Extension supported.
Virtualization enabled.

System Information <SOFTWARE>:
------------------------------
Computer Name: SIMONS-LAPTOP
OS Version : Windows 8 or greater
Server : No Server Version
--------------------------
Test Exception in TestCase
[Test OK] Result: (Expected: true == Result: true)

Test Exception Error Bad Ost
[Test OK] Result: (Expected: 0 == Result: 0)

Test Cmd Identifier
[Test OK] Result: (Expected: exec == Result: exec)

--------------------------
Executing OpenApp Command (mspaint)...
--------------------------
OpenApp started a new mspaint process
[Test OK] Result: (Expected: true == Result: true)

TerminateApp closed mspaint
[Test OK] Result: (Expected: false == Result: false)

Test Exception in TestCase
[Test OK] Result: (Expected: true == Result: true)

Test Exception for empty command
[Test OK] Result: (Expected: false == Result: false)
```

```

Test Exception for invalid app
[Test OK] Result: (Expected: false == Result: false)

Test Cmd Identifier
[Test OK] Result: (Expected: term == Result: term)

Test Exception in TestCase
[Test OK] Result: (Expected: true == Result: true)

TerminateApp setup: mspaint started
[Test OK] Result: (Expected: true == Result: true)

TerminateApp terminated mspaint
[Test OK] Result: (Expected: false == Result: false)

Test Exception in terminate flow
[Test OK] Result: (Expected: true == Result: true)

TerminateApp throws for invalid PID
[Test OK] Result: (Expected: false == Result: false)

TerminateApp throws for non-numeric PID
[Test OK] Result: (Expected: false == Result: false)

Menu::ShowMenu contains command identifier
[Test OK] Result: (Expected: true == Result: true)

Menu::ShowMenu contains usage text
[Test OK] Result: (Expected: true == Result: true)

Menu::ExecuteCommand calls Execute
[Test OK] Result: (Expected: true == Result: true)

Menu::ExecuteCommand passes arguments
[Test OK] Result: (Expected: arg1 arg2 == Result: arg1 arg2)

Menu::ExecuteCommand ignores unknown commands
[Test OK] Result: (Expected: false == Result: false)

Test Exception in TestCase
[Test OK] Result: (Expected: true == Result: true)

Menu::ShowMenu throws on bad ostream
[Test OK] Result: (Expected: 0 == Result: 0)

TEST OK!!
```

2.2.15 Testtreiber:

```

#include <fstream>
#include "OpenApp.hpp"
#include "TerminateApp.hpp"
#include "ListSystemInfo.hpp"
#include "vld.h"
#include "ListApps.hpp"
#include "Menu.hpp"
#include <sstream>
#include <iostream>
#include <cassert>
#include <vector>
#include <set>
#include <algorithm>
#include "Test.hpp"

#define WriteOutputFile ON
#define STR_EQUAL 0

using namespace std;

static bool TestListApps(ostream& ost);
static bool TestSystemInfo(ostream& ost);
static bool TestOpenApp(ostream& ost);
static bool TestTerminateApp(ostream& ost);
static bool TestMenu(ostream& ost);

static std::set<DWORD> GetProcessIdsByName(const std::string& exeName);
static bool ContainsProcessId(const std::set<DWORD>& pids, DWORD pid);


int main(void)
{

    ofstream output{ "Testoutput.txt" };

    if (!output.is_open()) {
        cerr << "Konnte␣Testoutput.txt␣nicht␣oeffnen" << TestCaseFail;
        return 1;
    }

    try {

                bool TestOK = true;


                TestOK = TestOK && TestListApps(cout);
                TestOK = TestOK && TestSystemInfo(cout);
                TestOK = TestOK && TestOpenApp(cout);
                TestOK = TestOK && TestTerminateApp(cout);
                TestOK = TestOK && TestMenu(cout);


        if (WriteOutputFile) {

                TestOK = TestOK && TestListApps(output);
                TestOK = TestOK && TestSystemInfo(output);
                TestOK = TestOK && TestOpenApp(output);
                TestOK = TestOK && TestTerminateApp(output);
                TestOK = TestOK && TestMenu(output);



            if (TestOK) {
                output << TestCaseOK;
            }
            else {
                output << TestCaseFail;
            }

            output.close();
        }

        if (TestOK) {
            cout << TestCaseOK;
        }
        else {
            cout << TestCaseFail;
        }
    }
    catch (const string& err) {
        cerr << err << TestCaseFail;
    }
    catch (bad_alloc const& error) {
        cerr << error.what() << TestCaseFail;
    }
    catch (const exception& err) {
        cerr << err.what() << TestCaseFail;
    }
    catch (...) {
        cerr << "Unhandelt␣Exception" << TestCaseFail;
    }

    if (output.is_open()) output.close();

        try {
                Menu menu;
                menu.AddCommand(move(make_unique<ListApps>(cout)));
                menu.AddCommand(move(make_unique<TerminateApp>()));
                menu.AddCommand(move(make_unique<OpenApp>()));
                menu.AddCommand(move(make_unique<ListSystemInfo>(cout)));

                menu.ShowMenu(cout);

```

```
                string input = "";
                cout << "Enter a command (or 'exit' to quit):" << endl;
                while (input != "exit") {
                        cout << ">";

                        getline(cin >> ws, input); // read the rest of the line without leading whitespace

                        if (input != "exit") {
                                menu.ExecuteCommand(input);
                        }
                }
        }
        catch (...) {
                cerr << "An unexpected error occurred. Exiting." << endl;
        }

        return 0;
}

bool TestListApps(ostream& ost)
{
        assert(ost.good());

        bool TestOK = true;
        string error_msg = "";

        try {

                ListApps listAppsCmd(ost);

                const string CMD_ID = "lsa";

                TestOK = TestOK && check_dump(ost, "Test Cmd Identifier", CMD_ID, listAppsCmd.GetCmdIdentifier());

                ost << "--------------------------" << std::endl;
                ost << "Executing ListApps Command..." << std::endl;
                listAppsCmd.Execute("");
                ost << "--------------------------" << std::endl;

        }
        catch (const string& err) {
                error_msg = err;
        }
        catch (bad_alloc const& error) {
                error_msg = error.what();
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test Exception in TestCase", true, error_msg.empty());
        error_msg.clear();

        try {
                stringstream bad_ost;
                bad_ost.setstate(ios::failbit); // set the failbit to simulate a bad ostream

                ListApps listAppsCmd(bad_ost);

                listAppsCmd.Execute("");
        }
        catch (const string& err) {
                error_msg = err;
        }
        catch (bad_alloc const& error) {
                error_msg = error.what();
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test Exception Error Bad Ost", STR_EQUAL,strcmp(error_msg.c_str(), ListApps::
             ERROR_BAD_OSTREAM));
        error_msg.clear();

        return TestOK;
}

bool TestTerminateApp(ostream& ost)
{
        assert(ost.good());

        bool TestOK = true;
        string error_msg = "";

        try {
                TerminateApp termCmd;

                const string CMD_ID = "term";
                TestOK = TestOK && check_dump(ost, "Test Cmd Identifier", CMD_ID, termCmd.GetCmdIdentifier());
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test Exception in TestCase", true, error_msg.empty());
        error_msg.clear();

```

```
        try {
                const auto before = GetProcessIdsByName("mspaint.exe");

                OpenApp openAppCmd;
                openAppCmd.Execute("mspaint");
                Sleep(750);

                const auto after = GetProcessIdsByName("mspaint.exe");
                std::vector<DWORD> newPids;
                for (DWORD pid : after)
                {
                        if (!ContainsProcessId(before, pid))
                        {
                                newPids.push_back(pid);
                        }
                }

                TestOK = TestOK && check_dump(ost, "TerminateApp␣setup:␣mspaint␣started", true, !newPids.empty());

                if (!newPids.empty())
                {
                        TerminateApp termCmd;
                        termCmd.Execute(std::to_string(newPids.front()));
                        Sleep(750);

                        const auto afterTerminate = GetProcessIdsByName("mspaint.exe");
                        TestOK = TestOK && check_dump(ost, "TerminateApp␣terminated␣mspaint", false, ContainsProcessId(afterTerminate
                                , newPids.front()));
                }
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt␣Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test␣Exception␣in␣terminate␣flow", true, error_msg.empty());
        error_msg.clear();

        try {
                TerminateApp termCmd;
                termCmd.Execute("0");
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt␣Exception";
        }

        TestOK = TestOK && check_dump(ost, "TerminateApp␣throws␣for␣invalid␣PID", false, error_msg.empty());
        error_msg.clear();

        try {
                TerminateApp termCmd;
                termCmd.Execute("abc"); // converts to 0
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt␣Exception";
        }

        TestOK = TestOK && check_dump(ost, "TerminateApp␣throws␣for␣non-numeric␣PID", false, error_msg.empty());
        error_msg.clear();

        return TestOK;
}

bool TestMenu(ostream& ost)
{
        assert(ost.good());

        bool TestOK = true;
        string error_msg = "";

        class DummyCommand : public Command
        {
        public:
                void Execute(const std::string& cmdArg) override
                {
                        mExecuted = true;
                        mLastArg = cmdArg;
                }

                std::string GetCmdName() const override
                {
                        return std::string(GetCmdIdentifier()) + "␣->␣Dummy␣(Usage:␣dmy␣<args>)";
                }

                std::string GetCmdIdentifier() const override
                {
                        return "dmy";
                }

                bool mExecuted = false;
                std::string mLastArg;
        };

        try {
                Menu menu;
                auto dummy = std::make_unique<DummyCommand>();
                auto* dummyPtr = dummy.get();
                menu.AddCommand(std::move(dummy));

```

```
                std::stringstream out;
                menu.ShowMenu(out);

                const std::string menuText = out.str();
                TestOK = TestOK && check_dump(ost, "Menu::ShowMenu␣contains␣command␣identifier", true, menuText.find("dmy") != std::string::npos);
                TestOK = TestOK && check_dump(ost, "Menu::ShowMenu␣contains␣usage␣text", true, menuText.find("Usage") != std::string::npos);

                menu.ExecuteCommand("dmy␣arg1␣arg2");
                TestOK = TestOK && check_dump(ost, "Menu::ExecuteCommand␣calls␣Execute", true, dummyPtr->mExecuted);
                TestOK = TestOK && check_dump(ost, "Menu::ExecuteCommand␣passes␣arguments", std::string("arg1␣arg2"), dummyPtr->mLastArg);

                dummyPtr->mExecuted = false;
                dummyPtr->mLastArg.clear();
                menu.ExecuteCommand("unknown␣something");
                TestOK = TestOK && check_dump(ost, "Menu::ExecuteCommand␣ignores␣unknown␣commands", false, dummyPtr->mExecuted);
        }
        catch (const string& err) {
                error_msg = err;
        }
        catch (bad_alloc const& error) {
                error_msg = error.what();
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt␣Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test␣Exception␣in␣TestCase", true, error_msg.empty());
        error_msg.clear();

        try {
                Menu menu;
                std::stringstream bad_ost;
                bad_ost.setstate(ios::failbit);
                menu.ShowMenu(bad_ost);
        }
        catch (const std::runtime_error& err) {
                error_msg = err.what();
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt␣Exception";
        }

        TestOK = TestOK && check_dump(ost, "Menu::ShowMenu␣throws␣on␣bad␣ostream", STR_EQUAL, strcmp(error_msg.c_str(), Menu::ERROR_BAD_OSTREAM));
        error_msg.clear();

        return TestOK;
}

bool TestSystemInfo(ostream& ost)
{
        assert(ost.good());

        bool TestOK = true;
        string error_msg = "";

        try {

                ListSystemInfo listSysCmd(ost);

                const string CMD_ID = "lss";

                TestOK = TestOK && check_dump(ost, "Test␣Cmd␣Identifier", CMD_ID, listSysCmd.GetCmdIdentifier());

                ost << "--------------------------" << std::endl;
                ost << "Executing␣ListSysInfo␣Command..." << std::endl;
                listSysCmd.Execute("");
                ost << "--------------------------" << std::endl;

        }
        catch (const string& err) {
                error_msg = err;
        }
        catch (bad_alloc const& error) {
                error_msg = error.what();
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt␣Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test␣Exception␣in␣TestCase", true, error_msg.empty());
        error_msg.clear();

        try {
                stringstream bad_ost;
                bad_ost.setstate(ios::failbit); // set the failbit to simulate a bad ostream

                ListSystemInfo listSysCmd(bad_ost);

                listSysCmd.Execute("");
        }
        catch (const string& err) {
                error_msg = err;
        }
        catch (bad_alloc const& error) {
```

```
                error_msg = error.what();
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test Exception Error Bad Ost", STR_EQUAL, strcmp(error_msg.c_str(), ListSystemInfo::
             ERROR_BAD_OSTREAM));
        error_msg.clear();

        return TestOK;
}

bool TestOpenApp(ostream& ost)
{
        assert(ost.good());

        bool TestOK = true;
        string error_msg = "";

        try {
                OpenApp openAppCmd;

                const string CMD_ID = "exec";
                TestOK = TestOK && check_dump(ost, "Test Cmd Identifier", CMD_ID, openAppCmd.GetCmdIdentifier());

                const auto before = GetProcessIdsByName("mspaint.exe");

                ost << "--------------------------" << std::endl;
                ost << "Executing OpenApp Command (mspaint)..." << std::endl;
                openAppCmd.Execute("mspaint");
                ost << "--------------------------" << std::endl;

                Sleep(750);
                const auto after = GetProcessIdsByName("mspaint.exe");

                std::vector<DWORD> newPids;
                newPids.reserve(after.size());
                for (DWORD pid : after)
                {
                        if (!ContainsProcessId(before, pid))
                        {
                                newPids.push_back(pid);
                        }
                }

                TestOK = TestOK && check_dump(ost, "OpenApp started a new mspaint process", true, !newPids.empty());

                if (!newPids.empty())
                {
                        TerminateApp termCmd;
                        termCmd.Execute(std::to_string(newPids.front()));
                        Sleep(750);

                        const auto afterTerminate = GetProcessIdsByName("mspaint.exe");
                        TestOK = TestOK && check_dump(ost, "TerminateApp closed mspaint", false, ContainsProcessId(afterTerminate,
                             newPids.front()));
                }

        }
        catch (const string& err) {
                error_msg = err;
        }
        catch (bad_alloc const& error) {
                error_msg = error.what();
        }
        catch (const exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test Exception in TestCase", true, error_msg.empty());
        error_msg.clear();

        try {
                OpenApp openAppCmd;
                openAppCmd.Execute("");
        }
        catch (const std::invalid_argument& err) {
                error_msg = err.what();
        }
        catch (const std::exception& err) {
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt Exception";
        }

        TestOK = TestOK && check_dump(ost, "Test Exception for empty command", false, error_msg.empty());
        error_msg.clear();

        try {
       OpenApp openAppCmd;
                openAppCmd.Execute("__this_application_should_not_exist__");
        }
 catch (const std::runtime_error& err) {
                error_msg = err.what();
        }
        catch (bad_alloc const& error) {
                error_msg = error.what();
        }
        catch (const exception& err) {
```

```
                error_msg = err.what();
        }
        catch (...) {
                error_msg = "Unhandelt Exception";
        }

    TestOK = TestOK && check_dump(ost, "Test Exception for invalid app", false, error_msg.empty());
        error_msg.clear();

        return TestOK;
}

static std::set<DWORD> GetProcessIdsByName(const std::string& exeName)
{
        std::set<DWORD> pids;

        HANDLE hProcessSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
        if (hProcessSnap == INVALID_HANDLE_VALUE)
        {
                return pids;
        }

        PROCESSENTRY32 entry{};
        entry.dwSize = sizeof(entry);

        BOOL ok = Process32First(hProcessSnap, &entry);
        while (ok)
        {
                if (exeName == entry.szExeFile)
                {
                        pids.insert(entry.th32ProcessID);
                }
                ok = Process32Next(hProcessSnap, &entry);
        }

        CloseHandle(hProcessSnap);
        return pids;
}

static bool ContainsProcessId(const std::set<DWORD>& pids,DWORD pid)
{
        return pids.find(pid) != pids.end();
}
```